Team Orienteering Problem: A Comprehensive Algorithm Analysis

Adnan Ibney Faruq (2005007)
Fairuz Mubashwera (2005030)
M.M. Nayem (2005078)
Ninad Nobo (2005080)
Mushfiqur Rahman (2005107)
Arnab Dey Kabya (2005112)

Algorithm Course Project
CSE-462

January 26, 2026

Outline

1. Problem Definition
2. Complexity Analysis
3. Existing Algorithms
4. Selected Algorithms
    1. Branch and Price Algorithm
    2. ILS Algorithm
5. Experimental Design
6. Application
7. Conclusion and Future Work

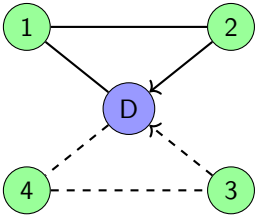
Team Orienteering Problem (TOP)

Definition

The **Team Orienteering Problem (TOP)** is a combinatorial optimization problem that involves three key decisions: *selection* (Knapsack), *routing* (TSP), and *assignment* to vehicles.

Problem Setting

- Customers with associated profits
- Multiple vehicles from a common depot
- Limited travel time per vehicle
- Not all customers can be visited

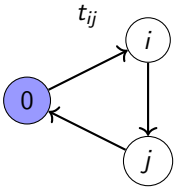


Key Constraint: Each customer is visited by at most one vehicle.

Formal Definition

Input

- Graph $G = (V, A)$; Depot 0; Customers C
- Travel times t_{ij} and Profits $p_i \geq 0$
- Vehicles m; Time limit $T_{\max}$



Vertices = depot + customers

Output: m routes maximizing total profit.

Objective

Goal

Maximize the total profit collected by all vehicles while respecting time constraints.

Objective Function

$$\max \sum_{i \in C} p_i \cdot x_i$$

where $x_i = 1$ if customer i is visited, 0 otherwise.

Note: Unlike TSP (visit all), TOP selects a profitable subset within a fixed time budget.

Feasibility Constraints

A solution is feasible if:

- Each route starts and ends at the depot.
- Each customer is visited by at most one vehicle.

Time Constraint

$$\sum_{(i,j)\in \text{route}_k} t_{ij} \leq T_{\max}, \quad \forall k = 1, \ldots, m$$

Note: A customer may remain unvisited if the travel cost exceeds the profit potential.

Illustrative Example

Instance

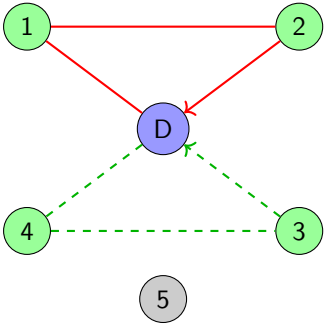
$n = 5, m = 2, T_{\max} = 10$
$p = [25, 40, 30, 35, 20]$

Optimal Solution:

- V1: $\{1, 2\}$ (Profit: 65)
- V2: $\{4, 3\}$ (Profit: 65)

Total Profit = 130

Customer 5 is skipped (low profit).



Complexity: Why is TOP Hard?

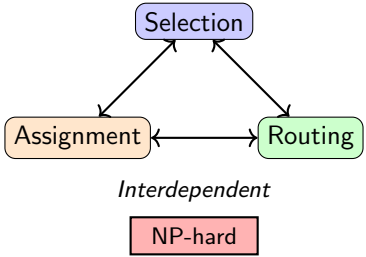
Interdependent Subproblems

TOP combines three NP-hard problems: **Selection** (Knapsack), **Routing** (TSP), and **Assignment**.

Three-Way Interdependence

- **Selection** $\leftrightarrow$ **Routing**: Routes determine node feasibility.
- **Assignment** $\leftrightarrow$ **Selection**: Fleet allocation changes profit limits.

$\Rightarrow$ **Cannot solve independently!**



NP-Hardness: Reduction from OP

Key Idea

The **Orienteering Problem (OP)** is a special case of TOP with only one vehicle. If we can solve TOP, we can solve OP. Since OP is already NP-hard, TOP must be too.

Construction

Given any OP instance:

Build a TOP instance by setting:

- $m = 1$ (single vehicle)
- Keep graph, profits, travel times
- Keep time budget $T_{\max}$ and target B

This takes constant time: $O(1)$

Why This Works

If OP has a solution:
That route is valid for TOP (with $m = 1$)

If TOP has a solution:
The single route solves the original OP

OP is NP-hard (Golden et al., 1987) $\Rightarrow$ **TOP is NP-hard**

Reduction to Maximum Coverage Problem

Key Idea

The **Maximum Coverage Problem (MCP)** selects k sets to maximize covered elements. TOP selects m routes to maximize profit. Same structure — different names.

Construction

Given any TOP instance:

Build an MCP instance by setting:

- Each feasible route $\rightarrow$ one set S_i
- Customers in route $\rightarrow$ elements of S_i
- Route profit $\rightarrow$ set weight w_i
- m vehicles $\rightarrow$ choose $k = m$ sets

Transformation is polynomial: $O(|R|)$

Why This Works

If TOP has a solution:
The m routes map to m sets with same profit

If MCP has a solution:
The k sets correspond to m valid routes

MCP is NP-hard $\Rightarrow$ **TOP is polynomially related to MCP**

Decision Version of TOP

Why Decision Version?

To prove NP-completeness, we need a YES/NO question, not an optimization problem.

Decision-TOP Problem

Given:

- A graph with depot and n customers
- Profits p_i and travel times t_{ij}
- Number of vehicles m and time limit $T_{\max}$
- **Target profit** B (this makes it a decision problem)

Question: Can we find m routes, each of length $\leq T_{\max}$, that together collect profit $\geq B$?

Next: We show this problem belongs to NP.

Proving TOP $\in$ NP

What Does "In NP" Mean?

A problem is in NP if we can **verify** a proposed solution quickly (polynomial time).

Certificate (Proposed Solution)

Someone gives us m routes:

- Route 1: Depot $\rightarrow$ customers $\rightarrow$ Depot
- Route 2: Depot $\rightarrow$ customers $\rightarrow$ Depot
- ... up to m routes

Size: At most $n \times m$ entries

Verification (Check in $O(n)$)

We verify three things:

1. Each route length $\leq T_{\max}$?
2. Same customer in multiple routes?
3. Total profit $\geq B$?

All checks are simple arithmetic!

Verification is polynomial $\Rightarrow$ **Decision-TOP** $\in$ **NP**

Conclusion: TOP is NP-Complete

Summary of Our Proof

1. **NP-hard:** We reduced OP to TOP (set $m = 1$)
   - OP is a known NP-hard problem
   - TOP is at least as hard as OP
2. **In NP:** Given a solution, we can verify it in polynomial time
   - Check route lengths, no duplicates, total profit

Final Result

Decision-TOP is NP-complete.

This means: No polynomial-time algorithm exists (unless $P = NP$).

Section Summary

Key Takeaways

- **Goal:** Maximize profit under time limits.
- **Challenge:** Mutual dependence of subproblems.
- **Difficulty:** NP-hard (Exact algorithms fail for large scale).

Transition

Because exact methods are exponential, we must utilize **Metaheuristics** for practical instances.

Next Section: Algorithmic Solution Approaches

Algorithm Classification

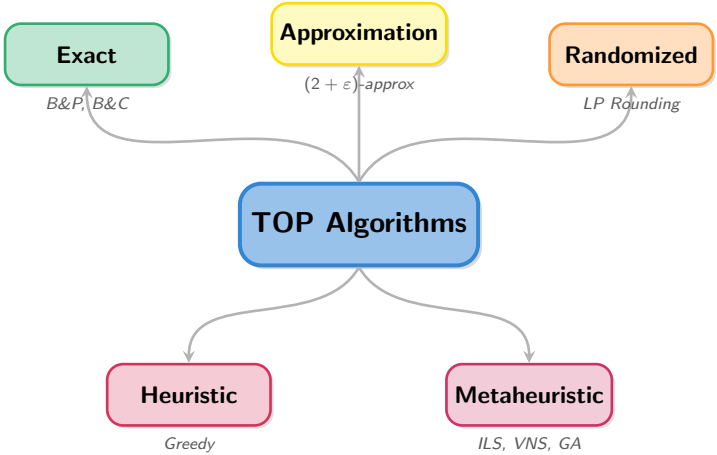


Figure: Algorithm Taxonomy for TOP

Algorithm Trade-offs

Exact Algorithms

- Guarantee optimal solutions
- Exponential time complexity
- Examples: Branch & Price, Branch & Cut

Approximation Algorithms

- **Guaranteed solution quality bounds:** The algorithm always produces a solution whose value is within a known factor of the optimal solution.
- **Polynomial time complexity:** The algorithm runs in polynomial time for any fixed choice of the approximation parameter.
- **Example:** $(2+\varepsilon)$**-approximation schemes:** For any chosen $\varepsilon > 0$, the algorithm guarantees a solution with profit at least $\frac{1}{2+\varepsilon}$ times the optimal profit. For instance, if the optimal profit is 100 and $\varepsilon = 0.1$, the algorithm guarantees a solution of at least 47.6.

Heuristic & Metaheuristic Algorithms

The Practical Choice

While they lack theoretical optimality guarantees, these methods are the standard for solving large-scale TOP instances due to their efficiency.

Key Characteristics

- **Efficiency**: Fast computation time even for large networks.
- **Performance**: Often produce near-optimal solutions in practice.
- **Trade-off**: No theoretical guarantees on the distance to the optimum.
- **Diverse Toolkit**: Includes simple methods (Greedy) and complex search strategies (ILS, VNS, GA).

Essential for real-world applications where "good enough" solutions are needed quickly.

Heuristic vs Metaheuristic Algorithms

Heuristic Algorithms

- **Idea**: Use simple, problem-specific rules to quickly build a solution.
- **Decision Style**: Make locally optimal choices (e.g., greedy selection).
- **Strength**: Extremely fast and easy to implement.
- **Limitation**: Can get trapped in poor local solutions.
- **Examples**: Greedy insertion, nearest-neighbor.

Metaheuristic Algorithms

- **Idea**: Provide a high-level search framework to improve solutions iteratively.
- **Decision Style**: Explore multiple neighborhoods and escape local optima.
- **Strength**: Produces high-quality solutions for large and complex instances.
- **Limitation**: More complex and requires parameter tuning.
- **Examples**: Iterated Local Search (ILS), Variable Neighborhood Search (VNS), Genetic Algorithms.

Randomized Algorithms

Core Idea

Randomized algorithms use controlled randomness to efficiently explore the solution space and avoid deterministic search traps.

Key Characteristics

- **Randomness**: Decisions are partially based on random choices.
- **Exploration**: Helps escape local optima and explore diverse solutions.
- **Efficiency**: Often faster than exact methods for large instances.
- **Variability**: Different runs may produce different solutions.
- **Guarantees**: May provide probabilistic or expected performance bounds.

Typical Techniques

- Randomized rounding of LP relaxations
- Random sampling and perturbation

Exact Exponential Algorithms

Overview

Exact methods seek the **provable optimal solution** by systematically exploring the search space. They are generally effective for instances with $n \leq 100$ nodes.

Algorithm	Description	Complexity	Solution Quality
Branch-and-Price	Based on **Column Generation**. Solves a set partitioning master problem via DP labeling.	Exponential	**Optimal** (State-of-the-art)
Branch-and-Cut	Uses a compact formulation with **Subtour Elimination** and cut generation.	Exponential	**Optimal** (Proven bounds)
Dynamic Prog.	Exploits submodular structure or state-space decomposition.	$O(n \cdot T_{\max} \cdot 2^n)$	**Optimal** (Small instances)
Integer LP	Formulates TOP as a **MIP** solved via LP relaxation and branching.	NP-Hard	**Optimal** ($n < 50$ nodes)

Meaning of "State-of-the-Art" and "Proven Bounds"

State-of-the-Art (Practical Perspective)

- Refers to the **best-performing known exact methods** in current research.
- Focuses on **computational efficiency** in practice.
- Typically able to solve **larger exact instances** compared to other exact approaches.
- Example: **Branch-and-Price** methods for TOP.

Proven Bounds (Theoretical Perspective)

- Emphasizes **mathematical guarantees** on solution optimality.
- Uses upper and lower bounds to **certify optimality**.
- Most effective for **small instances** ($n < 50$ nodes).
- Example: **Branch-and-Cut** and **Integer Linear Programming**.

Both approaches are exact, but emphasize different strengths: practice vs theory.

Computational Limitations of Exact Methods

The Curse of Dimensionality

While exact algorithms guarantee a provably optimal solution, their execution time grows **exponentially** with the number of customers n.

Applicability Thresholds

- **Small-Scale ($n \leq 100$)**: Generally solvable to optimality within minutes using state-of-the-art Branch-and-Price.
- **Medium-Scale ($100 < n < 300$)**: May require several hours and significant memory; often requires problem-specific cuts.
- **Large-Scale ($n \geq 300$)**: Search space becomes too vast for exact convergence.
- For medium to large-scale instances ($n > 100$),**non-exact algorithms** are preferred over exact methods.

Approximation Algorithms

Theoretical Guarantees

Approximation algorithms provide **polynomial-time** solutions with a **provable performance ratio** relative to the optimal solution.

Algorithm	Description	Complexity	Guarantee
Single-Vehicle OP	Recursive DP based on δ-excess k-TSP reduction.	$n^{O(1/\epsilon^2)}$	$(2+\epsilon)$
Greedy TOP	Iteratively solves single OP using residual submodular profits.	Polynomial	$1/(2+\epsilon)$
Euclidean PTAS	Exploits geometric partitioning in the 2D plane.	$n^{O(1/\epsilon)}$	$(1-\epsilon)$
FPT (Treewidth)	Dynamic programming on tree decompositions.	$2^{O(tw \log tw)} \cdot n$	**Optimal***

Theoretic Insights: Solvability vs. Approximation

Graphs with Bounded Treewidth

For graphs with a bounded treewidth (tw), the Team Orienteering Problem can be solved **optimally** in linear time relative to n.

General Graph Performance

For general graphs, the greedy framework provides a **constant-factor approximation**.

- **Approximation Ratio**: ≈ 0.33 (when $\epsilon = 0.5$).
- **Implication**: Guaranteed to find at least $1/3$ of the optimal profit in polynomial time.
- **Example:** If the optimal profit is 100, the greedy method is guaranteed to find a solution with profit at least 33.33.

This theoretical bound provides the baseline for evaluating more complex Metaheuristics.

Approximation Conventions: α vs. β

Two Equivalent Conventions

Researchers use two different but mathematically identical ways to report the approximation quality of a maximization problem like TOP.

Convention A: α-approx

Theoretical focus:

$$\text{OPT} \leq \alpha \cdot \text{ALG}$$

Common notation: $(2+\varepsilon)$-approximation.

Convention B: β-guarantee

Implementation focus:

$$\text{ALG} \geq \beta \cdot \text{OPT}$$

Common notation: $1/(2+\varepsilon)$ of optimal.

The Mathematical Identity

$$\text{OPT} \leq (2+\varepsilon) \cdot \text{ALG} \quad \Longleftrightarrow \quad \text{ALG} \geq \frac{1}{2+\varepsilon} \cdot \text{OPT}$$ **Example (OPT = 100, $\varepsilon = 0.1$):**

Both statements guarantee ALG ≥ 47.6.

Randomized Algorithms

Overview

Randomized approaches for TOP introduce controlled randomness in construction, rounding, or neighborhood exploration to cope with combinatorial complexity, often complementing deterministic heuristics.

Randomized Algorithms Overview

Algorithm	Description	Complexity	Solution Quality
LP Rounding	Solves LP relaxation using fractional values as selection probabilities.	Polynomial	Integrality gap dependent
Rand. TIM/RIS	Randomness in route insertion/improvement to diversify solutions.	Poly. per iteration	High-quality empirical results
Stochastic VNS	Randomized shaking to escape local optima.	Poly. per iteration	Effective (no guarantees)
Prob. SA	Probabilistic acceptance of worse moves via cooling schedule.	Heuristic	Near-optimal in practice

Note on Development

TOP lacks strong randomized approximation guarantees. Most success comes from embedding randomization within metaheuristics such as GRASP, VNS, SA, and GA.

Heuristic Algorithms

Overview

These algorithms utilize specific graph properties or iterative multi-step procedures to find high-quality solutions efficiently without a formal optimality guarantee.

Algorithm	Description	Complexity	Solution Quality
Simple Greedy	Iteratively inserts unvisited nodes at the best available position until infeasible.	$O(n^2 \cdot m)$	Variable (60-75%); fast baseline.
Profit-to-Cost	Ranks insertions by profit/time ratio to select the most efficient feasible options.	$O(n^2 \cdot m)$	Very good (75-85%); effective metric.
Chao et al. 5-Step	Two-phase approach: Greedy initialization followed by swap, 2-opt, and diversification.	Polynomial	High; robust multi-stage improvement.
Cheapest Insertion	Selects vertex-route pairs with minimum insertion cost (time increase per profit).	$O(n^2 \cdot m)$	Good (70-80%); balanced approach.

Meaning of "60–75% Solution Quality"

What the Percentage Represents

- Indicates the **empirical (observed) performance** of a heuristic algorithm.
- Measured by comparing the heuristic solution to the **optimal or best-known solution**.
- **Not a theoretical guarantee**; performance may vary across instances.

Illustrative Example

- Assume the optimal profit is **100**.
- A heuristic with **60–75%** solution quality typically finds solutions with profit between:

$$60 \text{ and } 75$$

- Example outcomes from different runs: 62, 68, 71, 74.

These percentages summarize average or typical performance observed in experiments, not worst-case guarantees.

Metaheuristic Overview

Search Strategy Focus

Advanced metaheuristics utilize hybrid frameworks to balance **intensification** (refining local areas) and **diversification** (exploring new areas).

Algorithm	Description	Complexity	Solution Quality
GRASP	Iterative construction using a Restricted Candidate List (RCL) followed by local search.	Polynomial	0.67% avg. gap (Chao benchmark).
LNS	A "destroy-repair" framework that removes and reinserts large node sets (q).	$O(q \cdot n \cdot m)$ per iter.	High; superior for large-scale instances.
GA	Evolutionary search using chromosome encoding and priority-list crossovers (OX/PMX).	Exponential (limit)	Best-known on 50% of standard instances.
ACO	Multi-agent pheromone-driven construction based on profit/time heuristic information.	Polynomial per ant	Strong in complex topologies.

Comparison of Algorithmic Approaches

The Performance Trade-off

Choosing an algorithm involves balancing the need for **Optimality** against the available **Time Budget** and **Problem Scale**.

Algorithm	**Type**	**Quality**	**Speed**
Branch-and-Price	Exact	Optimal	Slow (Hours)
$(2+\varepsilon)$-**Approx**	Approximation	Guaranteed	Fast (Poly)
Greedy Insertion	Heuristic	Variable	Very Fast
ILS	Metaheuristic	1–5% gap	Fast (Seconds)
VNS	Metaheuristic	1–3% gap	Fast (Seconds)
GA	Metaheuristic	Variable	Medium
ACO	Metaheuristic	Competitive	Medium
GRASP	Metaheuristic	0.67% gap	Fast
LNS	Metaheuristic	Competitive	Medium

$$\text{Gap } (\%) = \frac{\text{OPT} - \text{ALG}}{\text{OPT}} \times 100$$

What's Next

Transition to Implementation

While we have discussed a wide taxonomy of solvers, we will now analyze the mechanical details and performance of **two** representative algorithms:

Algorithm Selection

Selection Criteria

- **Algorithm 1**: Exact method for optimal solutions
- **Algorithm 2**: Metaheuristic for practical performance
- Complementary approaches: optimality vs. speed
- Well-documented and implementable

Algorithm 1:

- Branch-and-Price
- Exact algorithm
- Provides benchmark

Algorithm 2:

- Iterated Local Search
- Metaheuristic
- Fast and effective

Algorithm 1: Branch-and-Price

Why Branch-and-Price for TOP?

- **Proven effectiveness**: Gold standard exact method for vehicle routing problems; successfully applied to VRPTW, CVRP, and orienteering variants
- **Handles exponential route space**: Column generation adds routes on-demand, avoiding explicit enumeration of all feasible paths
- **Optimality guarantee**: Provides certified optimal solutions with proof of optimality, unlike heuristic approaches
- **Strong bounds**: Even when not solved to optimality, delivers tight dual bounds for benchmarking heuristics
- **Decomposition structure**: Separates route selection (master) from route generation (pricing), exploiting TOP's natural structure
- **Flexible extensions**: Easily incorporates additional constraints (time windows, precedence) through pricing subproblem modifications

Branch-and-Price: Implementation Architecture

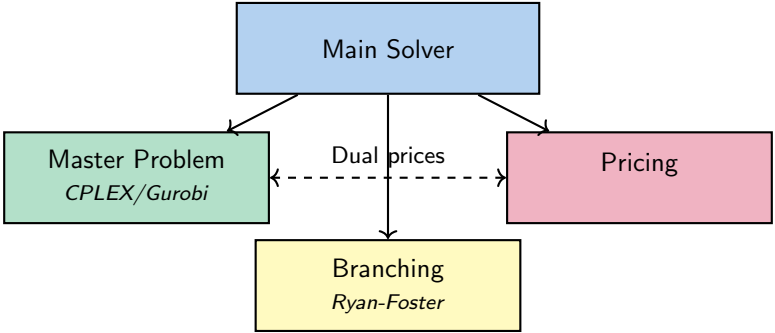


Figure: Implementation Architecture

LP Relaxation

Integer Program (IP)

$$\max \sum_{r \in \mathcal{R}} p_r x_r \quad \text{subject to:} \quad \sum_{r: i \in r} x_r \leq 1 \quad \forall i \in N, \quad x_r \in \{0, 1\}$$

LP Relaxation

$$\max \sum_{r \in \mathcal{R}} p_r x_r \quad \text{subject to:} \quad \sum_{r: i \in r} x_r \leq 1 \quad \forall i \in N, \quad 0 \leq x_r \leq 1$$

Why Relax?

- ✓ **Tractable:** Solvable in polynomial time
- ✓ **Upper Bound:** LP optimum $\geq$ IP optimum (for maximization)
- • **Trade-off:** Solution may be fractional $\Rightarrow$ need branching

Dual Prices

What Are Dual Prices (π_i)?

Shadow prices from the LP constraint: $\sum_{r:i\in r} x_r \leq 1 \quad \forall i \in N$

Interpretation: π_i = marginal value of visiting node i in current solution

Key Insight

$\pi_i > 0$: Node i is being used $\Rightarrow$ has value in current solution

$\pi_i = 0$: Node i is unused $\Rightarrow$ "free" to include in new routes

Role in Column Generation

Reduced profit: $\bar{p}_i = p_i - \pi_i$

$\Rightarrow$ Pricing subproblem uses reduced profits to find routes with $\bar{c}^* > 0$

$\Rightarrow$ If $\bar{c}^* \leq 0$, no improving column exists $\Rightarrow$ LP is optimal

Branch-and-Price: Main Algorithm

Branch-and-Bound with Column Generation

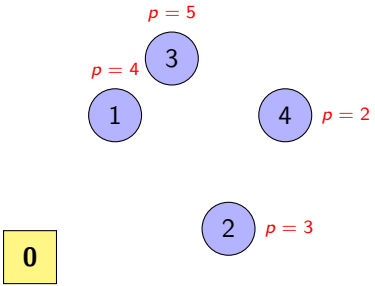
Input: Graph G, profits p, time limit T_{max}, m vehicles
Output: Optimal set of routes
Initialize root node with LP relaxation; Add initial columns
best_solution $\leftarrow \emptyset$; best_profit $\leftarrow 0$
while node queue is not empty **do**
  node $\leftarrow$ select_node() {Best-first or DFS}
  (columns, dual_prices) $\leftarrow$ column_generation(node)
  if LP_bound $\leq$ best_profit **then**
    prune node (bound)
  else if solution is integer **then**
    update best_solution if improved
  else
    branch on fractional variable
  end if
end while
return best_solution

Branch-and-Price: Column Generation

Column Generation Subroutine

```
Function column_generation(node):
while true do
    Solve restricted master problem (RMP)
    Get dual prices π for each vertex
    for each vehicle k = 1 to m do
        Solve pricing subproblem: p̃_i = p_i − π_i
        Find route with maximum reduced cost via DP
        if reduced cost > 0 then
            Add new column to RMP
        end if
    end for
    if no new columns added then
        break {Optimal LP solution found}
    end if
end while
return (solution, dual_prices)
```

B&P Simulation: Problem Instance



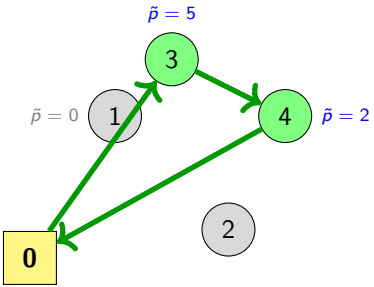
Instance

- $n = 4$ nodes, $m = 2$ vehicles
- Time limit: $T_{max} = 10$

Initial Columns

Route	Path	Profit
r_1	$0 \to 1 \to 0$	4
r_2	$0 \to 2 \to 0$	3

B&P Simulation: Column Generation



New: $0 \to 3 \to 4 \to 0$

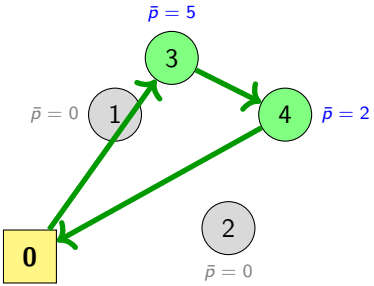
CG Iterations

Iter	LP	$\bar{c}^*$
1	7.0	7.0
2	12.0	2.0
3	14.0	0.0

Pricing Subproblem

- Get duals π_i from LP
- Reduced profit: $\tilde{p}_i = p_i - \pi_i$
- Find route with $\bar{c} > 0$ via DP

B&P Simulation: Column Generation (Iteration 1)



New: $0 \to 3 \to 4 \to 0$

Step A: Solve RMP

Initial routes: $r_1 : 0 \to 1 \to 0$, $r_2 : 0 \to 2 \to 0$

$$\max \; 4x_1 + 3x_2$$

Solution: $x_1 = 1, x_2 = 1$

LP = 7.0

Step B: Dual Prices

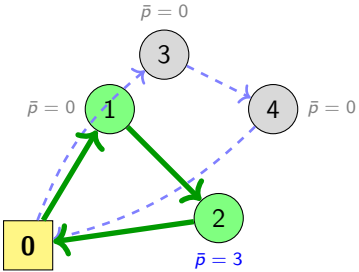
Node	1	2	3	4
π	4	3	0	0

Step C: Pricing Subproblem

Reduced profit: $\bar{p}_i = p_i - \pi_i$

Node	1	2	3	4
$\bar{p}$	0	0	**5**	**2**

B&P Simulation: Column Generation (Iteration 2)



New: $0 \to 1 \to 2 \to 0$

Existing: r_3

Step A: Solve RMP

Routes: r_1, r_2, r_3 ($0 \to 3 \to 4 \to 0$)

max $4x_1 + 3x_2 + 7x_3$

Solution: $x_1 = 1, x_2 = 0, x_3 = 1$, **LP = 12.0**

Step B: Dual Prices

Node	1	2	3	4
π	4	**0**	5	2

Node 2 unused $\Rightarrow \pi_2 = 0$

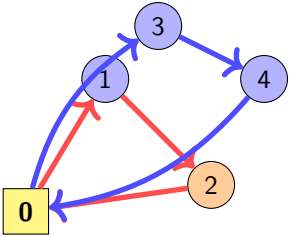
Step C: Pricing Subproblem

Reduced profit: $\bar{p}_i = p_i - \pi_i$

Node	1	2	3	4
$\bar{p}$	0	**3**	0	0

Best route: $0 \to 1 \to 2 \to 0$, $\bar{c}^* = 0 + 3 = \mathbf{3.0} > 0$ ✓

B&P Simulation: Column Generation Summary



V_1: $0 \to 1 \to 2 \to 0$

V_2: $0 \to 3 \to 4 \to 0$

Optimal Solution: 14 ✓

CG Iterations

Iter	LP	$\bar{c}^*$	Action
1	7.0	7.0	Add r_3
2	12.0	2.0	Add r_4
3	14.0	0.0	**Stop!**

Final Routes

Route	Path	Profit
r_3	$0 \to 3 \to 4 \to 0$	7
r_4	$0 \to 1 \to 2 \to 0$	7
	Total:	**14**

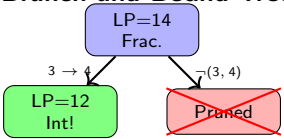
Key Insight

$\bar{c}^* = 0 \Rightarrow$ No improving route exists

Column generation complete!

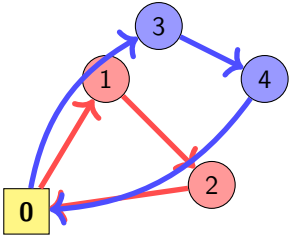
B&P Simulation: Branching & Solution

Branch-and-Bound Tree



Ryan-Foster branching on nodes 3, 4

Optimal Solution



V_1: $0 \to 1 \to 2 \to 0$ (profit 7)
V_2: $0 \to 3 \to 4 \to 0$ (profit 7)

Total: 14 ✓Optimal

B&P: Complexity Analysis

Master Problem (LP):

- Solve set partitioning/covering LP
- Variables grow with columns generated
- Complexity: $O(k^3)$ per solve, $k =$ columns

Pricing Subproblem (ESPPRC):

- Elementary shortest path with resource constraints
- DP labeling algorithm
- Complexity: $O(n^2 \cdot 2^n)$ worst case
- With dominance: pseudo-polynomial

Branching:

- Ryan-Foster branching on node pairs
- Tree size: exponential worst case
- Pruning via LP bounds
- Complexity: $O(2^n)$ nodes worst case

Practical Performance

- $n \leq 50$: seconds to minutes
- $n \in [50, 100]$: minutes to hours
- $n > 100$: may not terminate, but provides strong bounds

Algorithm 2: Iterated Local Search

Why ILS for TOP?

- **Proven effectiveness**: Consistently ranks among top metaheuristics for vehicle routing problems in literature
- **Escape local optima**: Perturbation mechanism allows exploration beyond local minima, unlike pure local search
- **Solution structure preservation**: Unlike genetic algorithms, ILS maintains feasible route structures during search
- **Scalability**: Handles large instances ($n > 500$) where exact methods fail
- **Anytime behavior**: Returns valid solution at any point; quality improves with more time
- **Easy parameter tuning**: Few parameters (perturbation strength, iterations) vs. complex metaheuristics

ILS: Pseudocode Implementation

Main Loop

```
class ILS_TOP:
  def solve(self):
    current = self.greedy_construction()
    current = self.local_search(current)
    best = current.copy()

    for iter in range(max_iterations):
      perturbed = self.perturb(current, strength=0.2)
      improved = self.local_search(perturbed)

      if self.accept(improved, current):
        current = improved
        if improved.profit ¿ best.profit:
          best = improved

    return best
```

ILS: Local Search Operators

Insert Operator:

- Try inserting unvisited vertex
- Check all insertion positions
- Select best feasible insertion
- Complexity: $O(n^2 \cdot m)$

Delete Operator:

- Remove vertex from route
- Update route time and profit
- Complexity: $O(n \cdot m)$

Swap Operator:

- Exchange two vertices
- Intra-route or inter-route
- Complexity: $O(n^2 \cdot m^2)$

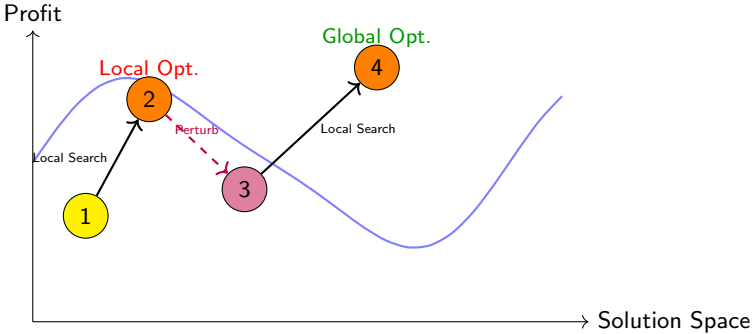
2-opt Operator:

- Remove two edges, reconnect
- Only within same route
- Complexity: $O(n^2 \cdot m)$

Implementation Strategy

- Use efficient data structures (adjacency lists)
- Cache distance calculations
- Early termination if no improvement

ILS: Visual Simulation

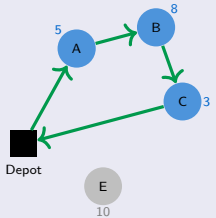


ILS Process (Step by Step)

1. **Start**: Generate initial solution (greedy construction)
2. **Local Search**: Climb to nearest local optimum
3. **Perturb**: Random shake to escape (jump to new region)
4. **Repeat**: Local search again → may find better optimum!

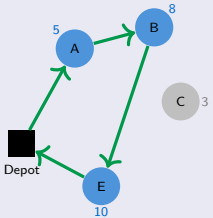
ILS: Route Example Simulation

Before Perturbation



Profit $= 5 + 8 + 3 =$ **16**

After Perturbation + LS



Profit $= 5 + 8 + 10 =$ **23** ✓

Key Insight

Perturbation removed C (low profit) and local search found E (high profit). New solution is better!

Experimental Design

Research Questions

1. **Solution Quality :** How does the solution quality of ILS compare to that of optimal Branch and Price solutions?
2. **Runtime Scalability :** What is the runtime scalability of both algorithms as the instance size grows?
3. **Parameter Sensitivity** How do algorithm parameters affect ILS performance?
4. **Instance Difficulty :** What makes some TOP instances harder than others?

Datasets and Setup

Standard Datasets

Dataset	Customers	Vehicles	Instances	Year
Chao Benchmark	32-102	2-4	157	1996
Dang Benchmark	102-401	2-4	82	2013
Vansteenwegen OP/TOP	50–300	1–4	120	2009–2015

SetUp

- RAM: 16-32 GB
- OS: Ubuntu 22.04 / Windows 11
- Language: Python 3.10+
- LP Solver: CPLEX 22.1 or Gurobi 11.0
- Libraries: NumPy, NetworkX, matplotlib, pandas

Performance Metrics

Solution Quality

- **Objective value**: Total profit collected
- **Gap to best-known solution (BKS)**:

$$\text{Gap}(\%) = 100 \times \frac{\text{BKS} - \text{Canculated Solution}}{\text{BKS}}$$

- **Gap to optimal** (when B&P terminates):

$$\text{Gap}(\%) = 100 \times \frac{\text{Optimal Solution} - \text{Calculated Solution}}{\text{Optimal Solution}}$$

Performance Metrics

Computational Efficiency

- **CPU time** (seconds)
- **Iterations to convergence** (for ILS)
- **Number of LP solves** (for B&P)
- **Number of labels generated** (for DP pricing)

Robustness

- Success rate(%) of instances solved to optimality (B&P)
- Average gap over multiple runs (ILS)
- Standard deviation of objective (ILS with different seeds)

Experiment 1: Comparative Analysis

Setup

- **Instances**: All Chao benchmark ($n \leq 102$)
- **Algorithms**: B&P (exact), ILS
- **Time limit**: 3 hours for B&P, 15 minutes for ILS
- **Replications**: 1 run for B&P, 10 runs for ILS

Metrics

- Gap to optimum
- CPU time
- Solution quality distribution
- Success rate (optimality for B&P)

Experiment 2: Scalability Analysis

Setup

- **Instances**: Dang benchmark ($n \in [102, 401]$)
- **Algorithms**: B&P (up to $n = 200$) and ILS
- **Analysis**: Plot runtime vs. instance size

Expected

- B&P exhibits exponential growth: $T(n) \approx c \cdot 2^{n/k}$
- ILS exhibits polynomial growth: $T(n) \approx c \cdot n^{\alpha}$
- Estimate growth rates via regression

Experiment 2 : Scalability Analysis

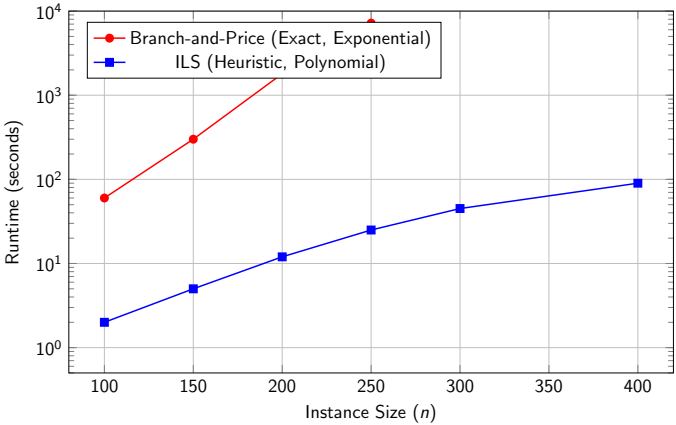


Figure: Expected runtime scalability of exact and heuristic methods for the Team Orienteering Problem (semi-log scale).

Experiment 3: Parameter Sensitivity (ILS)

Parameters

- **Perturbation strength**: $\{0.1, 0.2, 0.3, 0.4\}$
- **Max iterations**: $\{500, 1000, 2000, 5000\}$
- **Local search depth**: $\{1, 2, 3\}$ (number of neighborhoods)

Design

- Full factorial design: $4 \times 4 \times 3 = 48$ configurations
- Instances: Random subset of 20 instances
- Analysis: ANOVA to identify significant parameters
- Visualization: Heatmaps for parameter interactions

Experiment 4: Instance Difficulty Analysis

Instance Features

- **Graph density**: Average degree
- **Profit distribution**: Standard deviation and skewness
- **Time budget tightness**: $T_{\max}/\text{TSP_length}$
- **Spatial structure**: Cluster coefficient
- **Number of vehicles**: m

Analysis

- Correlation between features and:
  - B&P runtime
  - ILS gap to optimum
- Linear regression: Difficulty $= f(\text{features})$
- Identify most predictive features

Expected Results Summary

Expected Findings

- ILS achieves near-optimal solutions in minutes
- B&P solves most small-medium instances to optimality
- Runtime gap increases exponentially with instance size
- Perturbation strength is within a defined optimal for ILS
- Primary difficulty factor can be easily marked

Applications of the Team Orienteering Problem

Why TOP Matters

The Team Orienteering Problem (TOP) models decision-making scenarios where:

- Time and resources are limited
- Multiple agents operate simultaneously
- Not all locations can be visited
- Locations have different importance or profit

Key Application Domains

- Tourism and itinerary planning
- Logistics and technician routing
- Robotics and autonomous systems
- Disaster relief and humanitarian logistics
- Telecommunications and data collection

Application 1: Tourism & Itinerary Planning

Problem & Mapping to TOP

- Tourist wants to visit best places within limited time
- Vertices $\rightarrow$ Points of Interest (POIs)
- Profit $\rightarrow$ Tourist preference score
- Time $\rightarrow$ Travel + visit duration
- Vehicles $\rightarrow$ Days of the trip

Extensions:

- Time windows (opening hours)
- Mandatory attractions
- Category constraints (museum, food, nature)

Impact

- Used in travel recommendation apps
- 20–30% higher tourist satisfaction
- Planning time reduced from hours to seconds

Tourism Itinerary Planning as TOP

Conceptual Mapping

- Points of Interest (POIs) → Graph vertices
- Tourist preference → Profit
- Travel + visit duration → Cost
- Trip days → Vehicles

Points of Interest
↓
Preference Score
↓
Travel + Visit Time
↓
Trip Days

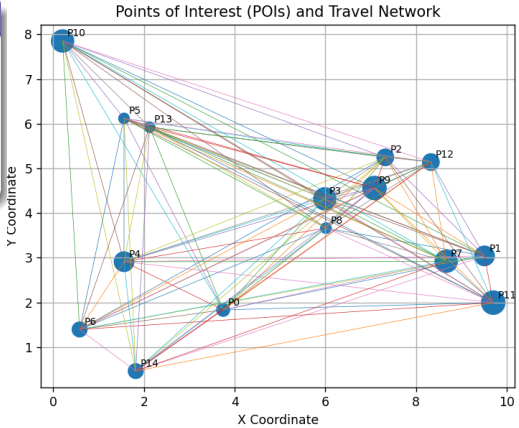


Figure: City modeled as POIs with travel costs

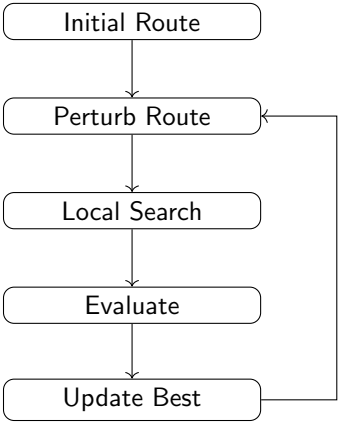
Optimization Approach: Iterated Local Search

Why Metaheuristics?

- Tourism TOP is NP-hard
- Exact methods do not scale
- Fast, high-quality solutions needed

Iterated Local Search (ILS)

- Start from feasible itinerary
- Perturb by inserting/removing POIs
- Apply local improvement
- Keep best solution



Simulation Results: Tourism Itinerary

Simulation Setup

- 15 POIs in a city
- Preference scores: 1–10
- Daily time budget: 8 hours
- Travel time based on distance

Performance Comparison

Method	Total Profit
Random Route	38
Greedy Heuristic	52
ILS Optimized Route	**68**

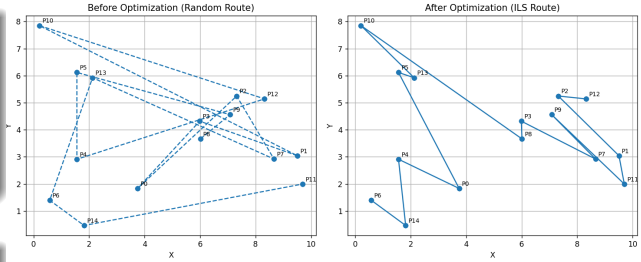


Figure: Route quality before and after optimization

Key Insight

Iterated Local Search significantly improves tourist satisfaction while respecting time constraints.

Application 2: Logistics & Robotics

Technician Routing

- Vertices $\rightarrow$ Customer requests
- Profit $\rightarrow$ Service priority
- Time $\rightarrow$ Travel + service time
- Constraint $\rightarrow$ Shift duration

Benefits:

- 15–25% higher service coverage
- Reduced overtime and operational costs

Multi-Robot Surveillance

- Vertices $\rightarrow$ Waypoints
- Profit $\rightarrow$ Information value
- Constraint $\rightarrow$ Battery life

Challenge: Dynamic and uncertain environments

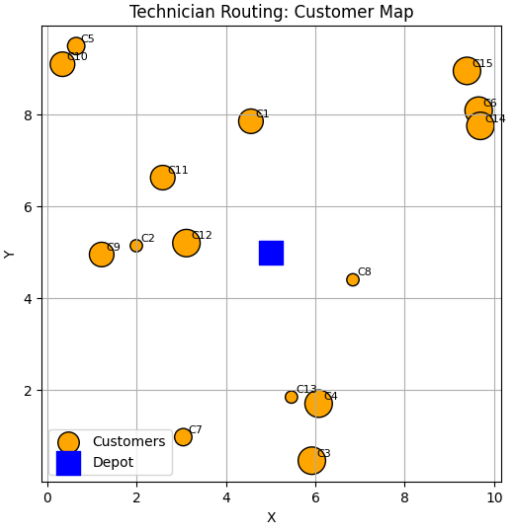
Logistics Application: Technician Routing

Mapping to Team Orienteering Problem

- Vertices → Customer service requests
- Profit → Service priority / urgency
- Travel cost → Distance between customers
- Service time → On-site repair duration
- Vehicle → Technician shift

Operational Constraints

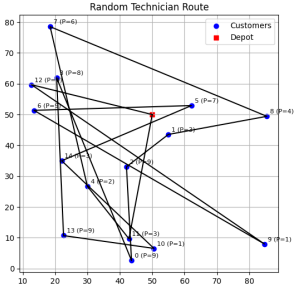
- Limited working hours
- Skill compatibility
- Mandatory high-priority jobs



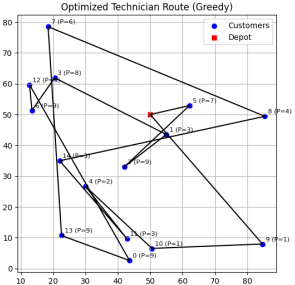
Technician Routing: Visual Simulation

Random vs Optimized Routing

TOP + ILS prioritizes high-value jobs while respecting shift duration constraints.



Random Assignment



Optimized (TOP + ILS)

Optimized routing reduces travel distance while serving higher-priority customers.

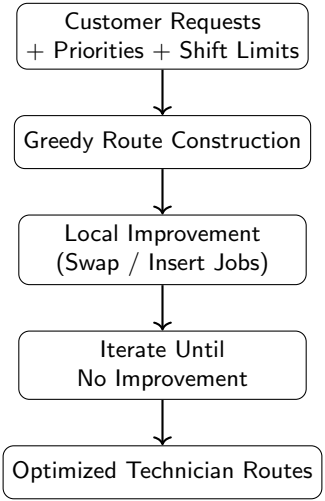
Solution Strategy for Technician Routing

Why Optimization is Needed

- NP-hard combinatorial routing problem
- Trade-off between service priority and travel time
- Real-time operational decision-making

Algorithm Used

- Greedy initial route construction
- Local search (swap / insert jobs)
- Iterated Local Search (ILS)



Simple optimization workflow for Technician Routing (TOP)

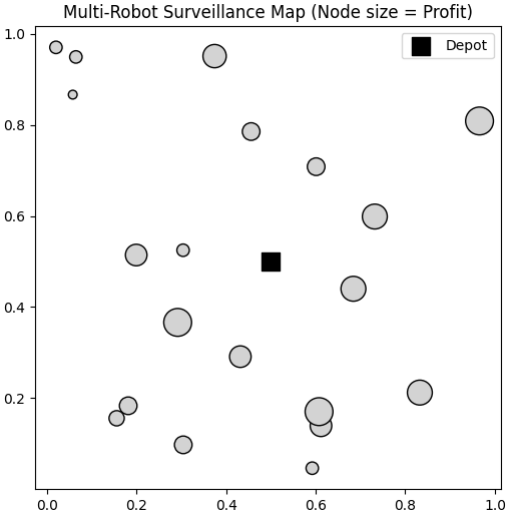
Robotics Application: Multi-Robot Surveillance

Mapping to Team Orienteering Problem

- Vertices → Surveillance waypoints
- Profit → Information value
- Cost → Travel distance
- Constraint → Battery capacity
- Vehicle → Autonomous robot

Challenges

- Dynamic environments
- Limited energy resources
- Partial observability

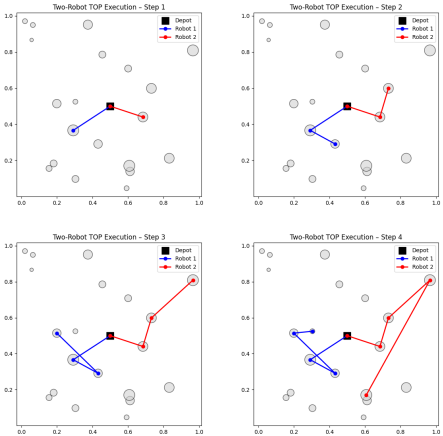


Surveillance waypoints and robot base

Robot Surveillance: Step-by-Step Execution (1–4)

TOP-Based Robot Path Construction

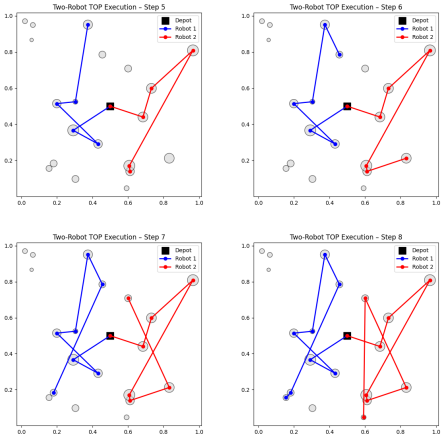
Robot starts from base and selects nearby high-value waypoints.



Robot Surveillance: Step-by-Step Execution (5–8)

Battery-Aware Path Completion

Robot avoids low-value distant waypoints as battery depletes.



Simulation Results & Practical Impact

Simulation Setup

- 10 technicians / robots
- 30 service requests / waypoints
- Limited shift duration or battery

Practical Impact

- 15–25% higher service coverage
- Reduced operational cost
- Increased robot autonomy

Performance Comparison

Method	Coverage (%)
Random Assignment	55
Greedy Heuristic	72
ILS + TOP Model	**89**

Key Insight

Team Orienteering enables intelligent decision-making under strict resource constraints.

Application 3: Disaster Relief

Emergency Resource Distribution

- Deliver aid after earthquakes, floods, or hurricanes
- Limited vehicles and strict time constraints

Mapping to TOP

- Vertices $\rightarrow$ Affected communities
- Profit $\rightarrow$ Urgency or people served
- Time $\rightarrow$ Travel + unloading
- Vehicles $\rightarrow$ Trucks / helicopters

Impact

- Improved coverage in first 72 hours
- Better prioritization of critical regions

Economic and Societal Impact

Economic Benefits

- 10–20% reduction in logistics costs
- Increased tourism revenue
- Efficient use of limited resources

Societal Impact

- Reduced fuel consumption and emissions
- Improved accessibility and fairness
- Faster emergency response

Key Takeaway

The Team Orienteering Problem successfully bridges theory and practice, delivering measurable real-world benefits.

Future Work

Algorithmic Improvements

- Develop faster and scalable heuristics for large instances
- Improve approximation guarantees
- Design efficient algorithms for special graph structures

Research Extensions

- Dynamic and stochastic versions of TOP
- Machine learning integration for better profit estimation
- Sustainability-aware and energy-constrained routing

Conclusion

Key Takeaways

- Team Orienteering Problem (TOP) models profit-maximizing routing under constraints
- Combines ideas from TSP and Knapsack problems
- Exact methods provide optimal solutions but scale poorly
- Heuristic and metaheuristic approaches give fast, near-optimal results

Practical Importance

- Used in tourism planning, logistics, robotics, and disaster relief
- Enables better resource utilization and decision-making
- Bridges theory and real-world optimization

References I

- Golden, B. L., Wang, Q., & Liu, L. (1987). A multifaceted heuristic for the orienteering problem. *Operations Research Letters*, 6(6), 361–368.
- Chao, I. M., Golden, B. L., & Wasil, E. A. (1996). The team orienteering problem. *European Journal of Operational Research*, 88(3), 464–474.
- Vansteenwegen, P., Souffriau, W., & Van Oudheusden, D. (2011). The orienteering problem: A survey. *European Journal of Operational Research*, 209(1), 1–10.
- Dang, D. C., Guibadj, R. N., & Moukrim, A. (2013). An effective PSO-inspired algorithm for the team orienteering problem. *European Journal of Operational Research*, 229(2), 332–344.
- Tricoire, F., Romauch, M., Doerner, K. F., & Hartl, R. F. (2012). Solving the team orienteering problem with a variable neighborhood search. *European Journal of Operational Research*, 217(3), 493–505.
- Gunawan, A., Marthinus Hendrik Setiawan, & Vansteenwegen, P. (2016). A hybrid genetic algorithm for the team orienteering problem. *European Journal of Operational Research*, 251(3), 735–748.